

Partiel de programmation fonctionnelle en Objective Caml

Christian Rinderknecht

1^{er} avril 2003

Nota : les questions sont indépendantes.

1. Écrire une fonction qui, étant donné $n \geq 1$, calcule u_n , où la suite $(u_n)_{n \geq 1}$ est définie par

$$u_1 = 1$$
$$u_{n+1} = \begin{cases} u_n/2 + 1 & \text{si } u_n \text{ est pair} \\ u_n + 17 & \text{sinon} \end{cases}$$

On prendra garde à ne pas faire un même calcul plusieurs fois.

2. Donner le type des expressions suivantes :

- `fun x -> x +. 3.0`
- `fun (x,y) -> x or y`
- `fun x -> let f y = y + x in f x`
- `List.map (fun x -> not x)`
- `fun f g -> (fun x -> f (g x))`

3. On désire manipuler symboliquement des expressions formées à partir des éléments suivants :

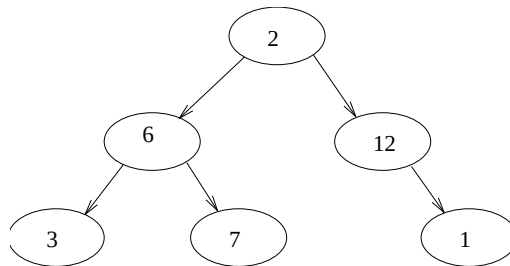
- la constante π ;
- des variables x , y , etc. ;
- un opérateur binaire $+$;
- un ensemble fixe de fonctions : $\sin$, $\cos$, et $\tan$.

Questions :

- Définir un type **expression** pour représenter ces expressions (les noms de variable peuvent être représentés par des chaînes de caractères) ;
- Donner la valeur Caml correspondant à l'expression $\cos x + \tan y$.
- Écrire une fonction **eval** qui calcule la valeur flottante d'une expression dans un environnement donné. On rappelle qu'un environnement est une liste de paires (nom de variable, valeur) qui à chaque variable associe une valeur. La fonction `List.assoc` sera utile pour accéder aisément aux contenu de l'environnement. (On

précise que les fonctions `sin`, `cos` et `tan` sont prédéfinies en Caml mais la constante π ne l'est pas : on utilisera la fonction arc-cosinus `acos`.)

4. On désire manipuler des polynômes à une variable et à coefficients entiers. On représentera un polynôme par une liste de couples (coefficient, degré), triée par ordre décroissant de degré ; chaque couple représente un monôme. Par exemple, $X^3 - X + 1$ sera représenté par la liste `[(1,3); (-1,1); (1,0)]`. Écrire une fonction qui effectue la somme de deux polynômes. On prendra garde à respecter l'ordre des monômes et à ne pas créer plusieurs monômes de même degré dans le résultat.
5. Un arbre binaire est un arbre où chaque nœud a au plus deux fils, un fils gauche et un fils droit. De plus, chaque nœud est annoté par une étiquette, qui est une valeur *a priori* quelconque. Voici un exemple d'arbre annoté par des étiquettes de type entier :



Le type des arbres binaires est défini par :

```

type 'a arbre =
  Rien
| Noeud of 'a * ('a arbre) * ('a arbre)

```

Ce type est *paramétré* par la variable de type `'a`, ce qui permet de définir le type des arbres indépendamment du type des étiquettes. On pourra ensuite utiliser le type `int arbre` lorsque l'on veut annoter chaque nœud par un entier, etc.

- Écrire une fonction qui calcule la hauteur d'un arbre.
- Écrire une fonction qui calcule la taille d'un arbre (c'est-à-dire le nombre de nœuds).
- Écrire une fonction `recherche`, qui prend en argument une étiquette `x` et un arbre `a`, et indique si cette étiquette apparaît sur l'un des nœuds de `a`.